IEcU Getting Started with Git & GitHub

Tony Gardella

2026-04-07

Table of contents

What is Git? 1
What is GitHub? 1
⚠️ Security & Best Practices at IEc 2
Installation & Setup 5
Core Concepts 6
Essential Git Commands 7
GitHub Basics 8
🎯 Follow Along Exercise: Add Your Name & Fun Fact 9
Quick Reference Cheat Sheet 13
Useful Software Tools for Git & GitHub 13

What is Git?

Git is a distributed version control system that tracks changes in your code over time. Think of it like a detailed "save history" for your project that allows you to:

- 📸 Take snapshots of your code at any point in time
- 🔄 Revert back to previous versions if something breaks
- 🤝 Collaborate with others without overwriting each other's work
- 🌿 Work on new features in isolation without affecting the main codebase

What is GitHub?

GitHub is a cloud-based platform that hosts Git repositories online. It adds a layer of collaboration tools on top of Git, including:

- A web interface to view your code and history
- **Pull Requests** — a way to propose and review changes before merging them
- Issue tracking, project boards, and team management
- A place to share and discover open source projects

💡 **How are they related?** Git is the software engine that enables version control on your computer/local network, GitHub is cloud/web tool that enables you to extend version control across multiple computers.

⚠️ Security & Best Practices at IEc

Before using Git and GitHub for any work at IEc, it is essential to understand the security responsibilities that come with it. Git is a powerful tool, but used carelessly it can expose sensitive data, client information, or proprietary code to the public internet.

Talk to Your Supervisor First

Before creating any repository or pushing any code to GitHub for a work project, speak with your supervisor. They will help you determine:

- Whether the project is appropriate for GitHub (public vs. private repositories)
- Whether the client has any restrictions on where code or data can be stored
- Whether IEc has an existing organizational GitHub account or workflow you should follow
- What level of access teammates and collaborators should have

🔴 **Important:** When in doubt, do not push. It is much easier to add files to a repository later than to scrub sensitive data from one that has already been made public.

Consult the Client

Many IEc projects involve sensitive client data, proprietary models, or confidential business information. Before using GitHub on a client project:

- **Review the contract or data use agreement** for any restrictions on data storage or sharing
- **Ask the client directly** if there are any concerns about hosting code on a third-party platform like GitHub
- Confirm whether the repository should be **private** and who is permitted to have access

Consult a Git Expert at IEc

If you are new to Git or unsure about how to structure a repository for a work project, **reach out to a Git-experienced colleague before getting started.** Getting the setup right from the beginning — branching strategy, access controls, what to include or exclude — is much easier than fixing mistakes after the fact.

The `.gitignore` File

One of the most important files in any Git repository is the **`.gitignore`** file. This file tells Git which files and folders to **intentionally ignore** and never track or commit.

Why It Matters

Once a file is committed and pushed to GitHub — especially a public repository — it is very difficult to fully remove. Even if you delete it in a later commit, it may remain in the repository's history. This is especially dangerous for:

- 🔑 **API keys and tokens** (e.g., OpenAI, AWS, Google Cloud)
- 🔐 **Passwords and credentials**
- 📁 **Environment variable files** (e.g., `.env`)
- 📊 **Sensitive or confidential data files** (e.g., `.csv`, `.xlsx` with client data)
- 🗂️ **Large or proprietary datasets**

How It Works

Create a file named `.gitignore` in the root of your repository. Each line specifies a pattern for files or folders to ignore.

```
# Example .gitignore

# Environment variable files
.env
.env.local
.env.*

# API keys or config files with secrets
secrets.R
config_private.yml

# Data files that should not be shared
data/raw/
*.csv
*.xlsx

# R-specific
.Rhistory
.RData
.Rproj.user/

# Python-specific
__pycache__/
*.pyc
.venv/

# OS-generated files
.DS_Store
Thumbs.db
```

Using a `.env` File for Secrets

A common best practice is to store all sensitive values — API keys, database passwords, tokens — in a `.env` file and load them into your scripts at runtime. **Never commit this file.**

Example `.env` file:

```
# .env  ← this file should NEVER be committed
API_KEY=your_secret_api_key_here
DB_PASSWORD=your_database_password
CLIENT_TOKEN=abc123xyz
```

In R, you can load these values using the `dotenv` or `usethis` packages:

```r
# Load environment variables from .env
dotenv::load_dot_env()

api_key <- Sys.getenv("API_KEY")
```

In Python, use the `python-dotenv` package:

```python
from dotenv import load_dotenv
import os

load_dotenv()

api_key = os.getenv("API_KEY")
```

> ✅ **Best Practice:** Add `.env` to your `.gitignore` before you write a single line of code. Make it the first thing you do when setting up a new repository.

GitHub's Secret Scanning

GitHub automatically scans public repositories for known secret patterns (API keys, tokens, etc.) and will alert you if one is detected. However, **do not rely on this as a safety net.** The best protection is to never commit secrets in the first place.

Security Checklist Before Your First Push

Before pushing a repository to GitHub — especially for the first time — run through this checklist:

- ☐ I have spoken with my supervisor about using GitHub for this project
- ☐ I have confirmed there are no client restrictions on code/data storage
- ☐ My repository is set to **Private** if it contains any sensitive or proprietary content
- ☐ I have created a `.gitignore` file and it includes `.env` and any data files
- ☐ I have checked that no API keys, passwords, or tokens are present in any file

- ☐ I have confirmed that no client data files are being tracked by Git (`git status`)

Installation & Setup

Step 1: Install Git

At IEc, ask CR to install Git.

For educational purposes, on your own non-IEc computer you could go to the Git download page, choose your operating system, and install it.

Step 2: Verify Installation

Open your Git terminal and run:

```
git --version
```

You should see something like: `git version 2.53.0` or something similair.

Step 3: Configure Git

Before you start using Git, tell it who you are. This information will be attached to every commit you make. Write these down so that we can use the same name and email for GitHub.

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Verify your configuration:

```
git config --list
```

Step 4: Create a GitHub Account

Go to https://github.com, click **Sign Up**, and follow the prompts using the `user.name` and email you configured above. Verify your email address.

Step 5: Authenticate with GitHub

To push code to GitHub, you need to authenticate. The recommended method is SSH keys.

Generate an SSH Key in your Git terminal:

```
ssh-keygen -t ed25519 -C "your.email@example.com"
```

Press Enter to accept the default file location and optionally set a passphrase.

Add the SSH Key to GitHub

Copy your public key:

```
cat ~/.ssh/id_ed25519.pub | clip
```

Go to **GitHub → Settings → SSH and GPG Keys → New SSH Key**, paste your key, and click **Add SSH Key**.

Test Your Connection:

```
ssh -T git@github.com
```

You should see: `Hi username! You've successfully authenticated...`

Core Concepts

Understanding these key concepts will make Git much easier to work with.

The Three States of Git

```
Working Directory  →  Staging Area  →  Repository
  (your files)        (git add)        (git commit)
```

- **Working Directory** — Where you edit files on your computer
- **Staging Area** — A holding area where you prepare changes before committing
- **Repository** — The permanent record of all your committed snapshots

Branches

A branch is an independent line of development. Think of it like a parallel universe for your code.

```
main branch:     A --- B --- C
                        \
feature branch:          D --- E
```

- The `main` branch is typically the stable, production-ready code
- You create a feature branch to work on something new
- When your work is ready, you merge it back into `main`

> ✅ **Best Practice:** Never work directly on `main`. Always create a new branch for your changes.

Essential Git Commands

Starting a Repository

```
# Initialize a new Git repo in the current folder
git init

# Clone (download) an existing repo from GitHub
git clone <repository-url>
```

Checking Status & History

```
# See the current state of your working directory and staging area
git status

# View commit history
git log

# Compact, one-line commit history
git log --oneline
```

Staging & Committing

```
# Stage a specific file
git add filename.txt

# Stage all changed files
git add .

# Commit staged changes with a message
git commit -m "Your descriptive commit message"
```

> 💡 **Write good commit messages!** A good message clearly describes what changed and why.

- ❌ Bad: `git commit -m "fix stuff"`
- ✅ Good: `git commit -m "Fix login button not responding on mobile"`

Branching

```
# List all local branches
git branch
```

```
# Create a new branch
git branch branch-name

# Switch to an existing branch
git checkout branch-name

# Create AND switch to a new branch (shortcut)
git checkout -b branch-name

# Modern syntax for switching branches
git switch branch-name

# Modern syntax for creating and switching
git switch -c branch-name

# Merge a branch into your current branch
git merge branch-name
```

Working with GitHub (Remote)

```
# View remote connections
git remote -v

# Add a remote repository
git remote add origin <repository-url>

# Push your branch to GitHub
git push origin branch-name

# Push and set upstream tracking (first push of a branch)
git push -u origin branch-name

# Pull latest changes from GitHub
git pull origin main
```

GitHub Basics

Key GitHub Concepts

Term	Definition
Repository (Repo)	Your project folder hosted on GitHub

Term	Definition
Fork	Your personal copy of someone else's repository
Clone	A local copy of a repository on your computer
Pull Request (PR)	A request to merge your branch into another branch
Merge	Combining changes from one branch into another
Issues	A way to track bugs, features, and tasks

The GitHub Workflow

This is the standard workflow used by professional teams:

1. Clone the repo → `git clone <url>`
2. Create a branch → `git checkout -b feature/my-feature`
3. Make your changes → *(edit files)*
4. Stage your changes → `git add .`
5. Commit your changes → `git commit -m "Add my feature"`
6. Push to GitHub → `git push -u origin feature/my-feature`
7. Open a Pull Request → *(on GitHub.com)*
8. Review & Merge → *(approved and merged on GitHub)*

🎯 Follow Along Exercise: Add Your Name & Fun Fact

Now it's time to put everything you've learned into practice! In this exercise, you will:

- Clone a shared repository
- Create your own branch
- Add your name and a fun fact to a file
- Commit your changes
- Push your branch to GitHub
- Open a Pull Request

Let's go!

Step 1: Clone the Repository

Open your terminal and clone the exercise repository:

```
cd ~/Documents

git clone git@github.com:tonygard-indecon/IEcU_github_tutorial.git
```

Navigate into the project folder:

```
cd IEcU_github_tutorial/git-workshop
```

Confirm you're on the `main` branch:

```
git status
```

Step 2: Create Your Own Branch

Create a new branch using your name. Use lowercase letters and hyphens — no spaces.

```
git checkout -b add/your-name
```

Example:

```
git checkout -b add/jane-doe
```

Verify you're on your new branch:

```
git branch
```

You should see your branch highlighted with an asterisk (*).

Step 3: Add Your Name & Fun Fact

Open the file called `participants.txt` in your text editor. It looks something like this:

```
# Workshop Participants
# Format: Name | Fun Fact
#
Example Person | I once ate 47 pancakes in one sitting 🥞
```

Add your own line at the bottom following the same format:

```
Your Name | Your fun fact here 🎉
```

Example entries:

```
Jane Doe | I have visited 23 countries and counting ✈
John Smith | I can solve a Rubik's cube in under 2 minutes 🧩
Alex Johnson | I once met my favorite author at a grocery store 📚
```

Save the file when you're done.

Step 4: Stage Your Changes

Check what changed:

```
git status
```

You should see `participants.txt` listed as modified. Stage the file:

```
git add participants.txt
```

Confirm it's staged:

```
git status
```

The file should now appear in green under "Changes to be committed".

Step 5: Commit Your Changes

Write a clear, descriptive commit message:

```
git commit -m "Add [Your Name] to participants list"
```

Example:

```
git commit -m "Add Jane Doe to participants list"
```

View your commit in the log:

```
git log --oneline
```

Step 6: Push Your Branch to GitHub

Push your branch up to the remote repository:

```
git push -u origin add/your-name
```

Example:

```
git push -u origin add/jane-doe
```

You should see output similar to:

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
...
To github.com:<org>/git-workshop.git
```

```
 * [new branch]  add/jane-doe -> add/jane-doe
Branch 'add/jane-doe' set up to track remote branch 'add/jane-doe' from 'origin'.
```

Step 7: Open a Pull Request on GitHub

1. Go to the repository on GitHub.com
2. You should see a yellow banner saying "Your branch had recent pushes" — click **"Compare & pull request"**
3. If you don't see the banner, click the **"Pull Requests"** tab → **"New Pull Request"**
4. Fill out the Pull Request form:
   - **Title:** `Add [Your Name] to participants list`
   - **Description:** Write a brief description of what you did (e.g., *"Added my name and fun fact to the participants file as part of the Git workshop exercise."*)
5. Click **"Create Pull Request"** ✅

Step 8: Review Someone Else's PR (Bonus!)

While you wait for your PR to be reviewed:

1. Go to the **Pull Requests** tab in the repository
2. Open a classmate's Pull Request
3. Click **"Files changed"** to see what they added
4. Leave a comment or an emoji reaction 👍
5. If everything looks good, you can **Approve** their PR

🎉 You Did It!

Once your PR is approved and merged, your name will be part of the project's `main` branch forever! Here's a summary of what you accomplished:

Step	Command Used
Clone the repo	`git clone`
Create a branch	`git checkout -b`
Check your changes	`git status`
Stage your changes	`git add`
Save your changes	`git commit -m`
Upload to GitHub	`git push`
Propose your changes	Pull Request on GitHub

Quick Reference Cheat Sheet

```
# ---- SETUP ----
git config --global user.name "Name"
git config --global user.email "email"

# ---- STARTING ----
git init                    # New local repo
git clone <url>             # Copy remote repo

# ---- DAILY WORKFLOW ----
git status                  # What's changed?
git add .                   # Stage everything
git add <file>              # Stage one file
git commit -m "message"     # Save a snapshot
git push origin <branch>    # Upload to GitHub
git pull origin main        # Download latest changes

# ---- BRANCHING ----
git branch                  # List branches
git checkout -b <name>      # Create + switch branch
git switch <name>           # Switch branch
git merge <branch>          # Merge into current branch

# ---- HISTORY ----
git log --oneline           # Compact history
git diff                    # See unstaged changes
```

Useful Software Tools for Git & GitHub

GUI Clients (Visual Git Interfaces)

```
GitKraken - Feature-rich, cross-platform GUI
Sourcetree - Free GUI by Atlassian
GitHub Desktop - Official GitHub client, beginner-friendly
Tower - Professional Git client (paid)
Fork - Fast and friendly Git client
```

Code Editors / IDEs with Git Integration

```
VS Code - Built-in Git support + GitLens extension
JetBrains IDEs (IntelliJ, PyCharm) - Excellent built-in Git tools
Sublime Merge - Git client by Sublime Text team
```

Productivity Extensions & Plugins

```
GitLens (VS Code) - Enhanced Git visualization
Git Graph (VS Code) - Visual branch/commit history
GitHub Copilot - AI code assistance
```

Code Review & Collaboration

```
Reviewable - Advanced code review tool
Codecov - Code coverage reporting
SonarQube - Code quality analysis
```

CI/CD Integration Tools

```
GitHub Actions - Built-in automation
Jenkins - Open-source automation server
CircleCI - Cloud-based CI/CD
Travis CI - Continuous integration service
```

Terminal Enhancements

```
Oh My Zsh - Git aliases and plugins
Git Bash - Unix-style terminal for Windows
Lazygit - Terminal UI for Git commands
diff-so-fancy - Improved diff output
```

Repository Management

```
BFG Repo Cleaner - Remove sensitive data from history
Git LFS - Large file storage
Husky - Git hooks made easy
```

Project Management (GitHub Adjacent)

```
ZenHub - Agile project management in GitHub
Jira - Issue tracking with GitHub integration
Linear - Modern issue tracking
```

Happy coding! Remember: every expert was once a beginner. The more you use Git, the more natural it becomes. 🌱